An Optimal Deterministic Adversary for Sorting

Nick Nash

7 November 2026

Initial draft for discussion

Abstract

This paper describes an optimal deterministic, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary recursively moves items into ordered subproblems, extending the classical framework in which items move down a binary tree. Within each subproblem it counts weakly order-preserving integer assignments and chooses comparison answers using exact counts on small local neighborhoods. Reusing child subproblems supplies factorial credits that offset the cost of keeping these local calculations tractable, leaving only $o(n \log n)$ additional loss.

Methodological disclosure

The novel ideas in this paper were generated by GPT 6 Astra, which also assisted with drafting and revising the exposition and LaTeX. All figures were drafted and revised as TikZ code with the same system. The human authors made all final scientific and editorial decisions and have substantially simplified and independently verified all results. We take full responsibility for all submitted content.

1 Introduction

The usual comparison lower bound for sorting is a counting argument: a binary decision tree distinguishing all $n!$ orders has a path of length at least $\lceil \log_2(n!) \rceil$. It does not immediately tell an adversary how to find such a path efficiently. An adversary must answer each comparison as it arrives, without knowing the sorter's future queries, and every answer must remain consistent with distinct keys in a strict total order.

An unrestricted adversary can always choose the answer retaining more linear extensions of the current partial order. The computational difficulty is that exactly counting linear extensions is #P-complete [3]. This does not rule out an efficient optimal adversary; it suggests looking for a different quantity to count.

Our adversary follows that route. It counts auxiliary integer assignments, permits ties in those assignments, and recursively preserves the freedom to resolve ties. Its organization extends the classical adversaries that move items down an ordered tree. The main difference is that a move is charged for the freedom it loses in its parent *and credited* for the freedom it creates in its child.

Theorem 1.1 (Main result)**.** *There is a single deterministic adversary which, initialized using only n, answers comparisons consistently with a strict total order on n distinct keys, uses a fixed polynomial number of bit operations per answer, and forces every correct comparison sorter to make*

$$(1 - o(1))n \log_2 n$$

comparisons.

Here ordinary sorting means comparison sorting of distinct, otherwise unrestricted keys. The sorter can choose any pair adaptively. The adversary does not receive a target accuracy or a description of the sorter. The coefficient is optimal since merge sort uses at most $n\lceil \log_2 n \rceil$ comparisons. We do not claim an additive $O(n)$ gap from $\log_2(n!)$, or useful finite-size thresholds.

1.1 The earlier adversaries

Atallah and Kosaraju. Atallah and Kosaraju gave an explicit sorting adversary [1]. One useful way to view its effect is through individual items: Richards and Vaidya describe how their technique forces, for each $1 \le k < n$, $n - k$ items to participate in at least $\log_2(k + 1)$ comparisons each [6, Section 2]. Each comparison has two participants. Taking, for example, $k = \lfloor n/\log_2 n \rfloor$ gives the leading coefficient $1/2$ for the total number of comparisons.

Borodin, Guibas, Lynch, and Yao. In their work on preprocessing for search, Borodin et al. gave a particularly transparent bin-tree adversary [2, Section 4]. Place all items at the root of an infinite ordered binary tree. For a query on x, y:

(i) If they occupy the same node, move x to its left child and y to its right child; answer $x < y$.

(ii) If one's node properly contains the other in its subtree, move the ancestor item to the child outside that subtree; answer according to the resulting left-to-right order.

(iii) If their nodes already lie in separate subtrees, answer according to their left-to-right order without moving either item.

The spatial commitments preserve all earlier answers. Items at the same node or at ancestor-related nodes remain publicly incomparable.

Brodal, Chaudhuri, and Radhakrishnan. Brodal et al. use this tree strategy in their work on maintaining rank information and explicitly derive its sorting bound [4, Sections 4–4.1]. At termination the occupied nodes are pairwise nonancestral and each contains one item. If their depths are $d_1, \ldots, d_n$, the disjoint binary subtrees give $\sum_i 2^{-d_i} \le 1$. The arithmetic–geometric mean inequality yields $\sum_i d_i \ge n \log_2 n$. A comparison increases the sum of depths by at most two, giving $q \ge \frac{1}{2} n \log_2 n$. This identifies the limitation of the depth accounting: a comparison can pay for two downward steps.

Richards and Vaidya also give a binary-tree formulation and a weighted-poset adversary [6, Sections 3–4]. In the latter, item weights are halved at comparisons while every chain has weight at most one. The terminal inequality $\sum_x 2^{-c(x)} \le 1$, with $c(x)$ the participation count, gives the same coefficient. Kaplan, Zamir, and Zwick subsequently improved the efficient deterministic guarantee to $37/64$ [5].

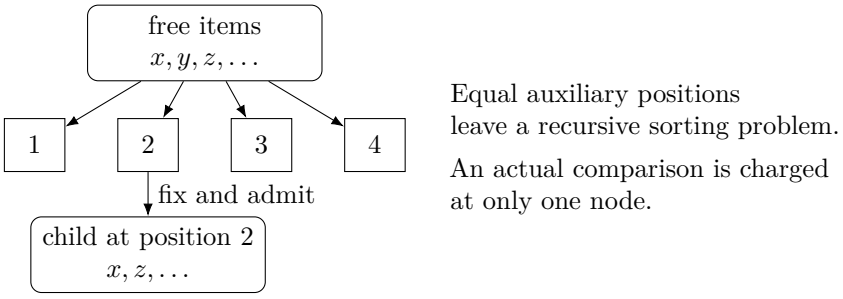


Figure 1: A move fixes an item's position in its parent and makes it free in the corresponding child. The order between items sharing that position is resolved recursively.

1.2 Our adversary in the moving-items framework

We retain three features of the bin-tree view: items begin together, a move commits an item to a more specific ordered region, and a comparison whose endpoints still share a region is handled recursively there. We replace the rule of separating the queried items immediately by retaining nearly half of a counted family, then moving items when needed to simplify that family.

At a node, the regions are an ordered list of integer positions. A free item can occupy an interval of these positions. Once assigned a fixed position, it enters the recursive child associated with that position. Several items can have the same parent position and remain unresolved in the child. The integer positions are private auxiliary data, not key values.

This is an *ordered multiway refinement* of the binary-tree framework. A node's destinations can be drawn as leaves of a binary tree, but the algorithm uses ordered lists. This expansion describes destinations; it does not preserve the old ancestor-incomparability invariant. Free items can already be publicly comparable, so we record their public order explicitly.

The quantity replacing total depth is

$$\log_2 Z, \qquad Z = \prod_v Z_v,$$

where Z_v counts the node's admissible weakly order-preserving integer assignments. Initially $Z = n^n$. At least one of the weak answers $X_x \le X_y$ and $X_y \le X_x$ retains at least half the count: the two events cover the family and overlap at ties. Small local calculations approximate the needed probabilities.

To make those calculations reliable we keep each active graph sparse, shallow, and controlled in its interval widths. Repairs can discard assignments. But the jth item admitted to a child creates at least j new choices there. Repeated admissions therefore supply a factorial credit. The proof shows that all net repair losses total $o(n \log n)$. At completion the auxiliary count may still exceed one, but its logarithm is only $O(n \log \log n)$. This suffices for the leading coefficient.

Organization and verification status. Section 2 gives the state, comparison procedure, and all repair rules before their proofs. Sections 3–6 prove legality, local computation, and the lower bound. The complete local approximation argument is in Appendix A. The accompanying repository contains formal proofs of components of this construction; it does not yet provide a complete Lean proof of Theorem 1.1, including its controller and bit complexity. The theorem here is supported by the mathematical argument below.

Notation. All logarithms have base two unless written ln. Put $T = n \log_2 n$ and interpret $0 \log_2 0$ as zero. The *public order* P is the transitive closure of actual comparison answers; $x \leq_P y$ includes $x = y$. All algorithmic choices below use fixed orders on labels, edges, paths, and blocks to break ties.

2 The mechanical adversary

We first specify one fixed stage of the algorithm. Its parameters are constants independent of the play. Section 6 gives the effective rule selecting an improving stage from n.

2.1 Stored state

A node v contains an ordered list of positions, an interval for every free item, fixed positions for the other items, and selected weak inequalities $X_x \leq X_y$. Position indices are $1, \ldots, K$; positions also have permanent identities, so insertion never changes the identity of an old fixed position. Every position with fixed items has a child containing exactly those items. Each original item is free at exactly one current node.

There are two kinds of record. The public order retains every actual answer. The smaller *active graph* retains selected inequalities between free items. Comparisons involving fixed items become interval bounds. Some active edges will be retired in favor of stronger interval bounds; their actual comparisons are never removed from P. We keep the underlying comparisons and fixed identities so their interval consequences can be recomputed after an insertion. Separately stored *artificial bounds* are intervals imposed by the adversary itself.

An assignment is admissible if it respects these intervals, fixed values, and inequalities. The stored conditions imply $X_x \leq X_y$ whenever $x <_P y$ among items present in the node. Give all admissible assignments equal weight; probabilities below always refer to this finite distribution. The algorithm need not compute Z_v globally.

Propagate lower bounds forward along inequalities and upper bounds backward. If $[a_x, b_x]$ are the stored intervals, the resulting effective intervals are

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x + 1, \tag{1}$$

where reachability includes the item itself and fixed values are singleton intervals. After propagation, fixed items can be eliminated from numerical calculations. Every free item must have $W_x \geq 2$. An item newly entering a child must be publicly incomparable with all items already there.

2.2 The primitive move: fix in the parent, admit in the child

To move x, choose a position c strictly inside its effective interval, fix $X_x = c$, and transfer x to the child at c as a new free item. Incident active edges are replaced by their interval consequences.

If the chosen position is the lower endpoint $c = \ell_x < r_x$, first insert a new position c^+ immediately after it. Enlarge each artificial interval containing c to include both copies. Old fixed items stay at the old identity c. Recompute comparison-induced bounds from their original inequalities, and fix x at c^+. At the upper endpoint use the reflected operation, inserting immediately before it. This preserves the cost bound for the originally selected endpoint; see Lemma 3.2.

A child begins with two positions. At its jth admission, append a top position if necessary to give it at least j positions. Leave all old interval endpoints unchanged, and give only the

incoming item the full current list. The incoming item has no comparisons with old child items, so this multiplies the child's count by at least j.

2.3 A reproducible choice of position

For ordinary moves we will obtain an interval I of m positions such that every $t \in I$ has probability at least $1/(16m)$. Certain repairs use a different constant V in place of 16. In either case the following rule chooses a position that can be reused.

Maintain binary-counter batches of positions: a position is born in a singleton batch, and equal-size batches merge until there is at most one of each size. A batch's members keep their relative position order for its lifetime, even if other positions are inserted between them. Within a batch, the aligned block of size 2^k with indices $j2^k + 1, \ldots, (j+1)2^k$ has representative its sole member if $k = 0$, and its member at index $j2^k + 2^{k-1}$ otherwise.

For $m \geq 8$, delete the first and last positions of I. Intersect the remaining interval with every current batch, decompose each intersection into maximal aligned blocks, and choose a largest block. Fix at its representative. For $m < 8$, use the first position of I and perform an endpoint insertion if needed. Record this exceptional choice as block size one. Initially present positions are entered into batches in their order.

A scale group below uses a restricted version. During that group a position cannot be chosen twice as a singleton-block representative, or twice as a larger-block representative. On later choices discard earlier chosen blocks of the same kind before taking a largest block. A position represents at most one larger block in a fixed batch. The width separation below ensures enough blocks remain. The group's parent list and batches stay unchanged.

2.4 Stage constants and local queries

Choose integers $D, H \geq 4$ and a rational $0 < \eta \leq 1/16$. Set

$$d = D + 1, \qquad h = 2H, \qquad M = (d+1)^{h+1}, \qquad B = 2M, \qquad C_* = 2 \cdot 4^B. \tag{2}$$

Choose integers $L \geq 4$, $J \geq 4$ with $C_*/L \leq \eta$, and set

$$A = 128J(L+1)^{2h+3}, \qquad R = A^{J+1}. \tag{3}$$

For these d, h, R, η, compute the radius ρ and width threshold W_0 from Appendix A. Increase W_0 to at least 32 if needed. Put $G = 4^B W_0$.

After repairs, active degree is less than D and active height (vertices on a longest directed path) is less than H. During repairs we may use the larger bounds d, h above. Each item then has at most B other active predecessors and successors.

A LocalQuery on one or two items retains their radius-ρ neighborhood in the active graph, together with the *globally propagated* intervals (1). It counts this finite restricted system exactly using the subset dynamic program in Section 4.1. It can return either a weak comparison probability or a coordinate cumulative probability. After width repairs these estimates have error at most η. An isolated item is an independent uniform coordinate and is handled exactly, at any width.

An OrdinaryMove finds the first crossings a, b of $1/4, 3/4$ in one such local cumulative distribution, applies the representative rule to $[a, b]$, and moves the item. Since $\eta \leq 1/16$, every position in that interval has true probability at least $1/(16(b - a + 1))$.

2.5 The repair procedure

The following rules act only in the node that has just answered a comparison. Recompute effective intervals after every change.

Width repairs. Repeatedly take the first applicable rule, ignoring isolated items.

W1. Narrow item. If $W_x < W_0$, let

$$I_x = \left[\ell_x + \left\lfloor \frac{W_x - 1}{4} \right\rfloor, r_x - \left\lfloor \frac{W_x - 1}{4} \right\rfloor\right].$$

Choose its first position, make it interior if needed, and move x. This is a *cheap move*; its parent retains at least $1/G$.

W2. An edge across unequal widths. For an edge $u \to v$ with $W_u > LW_v$, impose the artificial bounds $X_u \le \ell_v - 1$ and $X_v \ge \ell_v$, then retire the active edge. If $W_v > LW_u$, instead impose $X_u \le r_u$ and $X_v \ge r_u + 1$. These strict cuts preserve the answered comparison and retain at least $1 - \eta$ of the count.

W3. A component across too many scales. If a component's largest width exceeds R times its smallest width w, take a shortest path from a smallest-width item to a largest-width item. For $i = 0, \ldots, J - 1$, select the first item of width at least $A^i w$. Process the selected items in increasing width, choosing representatives from their central intervals I_x by the restricted group rule. These are *scale moves*. They use interior positions, insert no parent positions, and send at most two items to each child.

Return to W1 after each rule or completed group. The selected scale items retain their intervals and incident edges until processed.

Degree and path repairs. First finish all width repairs. If an item currently has degree at least D, perform an ordinary move of that item, repair widths, and retest degrees. Otherwise, find a longest directed path by topological dynamic programming. If it contains at least H free items, reserve its first H items. Process them in path order: before each ordinary move repair widths; skip reserved items already moved by those repairs. Move a reserved item even if it has since become isolated. Complete this reservation before choosing a new path. Repeat until degree is less than D and height is less than H.

No child is repaired merely because it receives an item: the new item is isolated there and its admission changes none of the old distributions.

2.6 Answering an actual comparison

An answer at a node is *safe* if it neither creates a new public relation between two old items fixed at the same position, nor reduces any free effective interval to a singleton, and its weak constraints remain feasible. Test this by public transitive closure and interval propagation under each proposed answer.

Only a comparison with two free endpoints adds an active edge; a fixed endpoint supplies a bound. Routing through ancestors changes none of their counts. Lemma 3.4 proves that the safe all-retaining branch always exists when needed.

The root starts with n positions, all n items free on the full interval, and no edges. For $n \le 1$ there is nothing to answer. If no stage fits the size caps of Section 6, use a fixed label order. After $2n\lceil \log_2(n+1) \rceil$ actual queries, the algorithm may fix any total order extending P and answer according to it thereafter. This final rule bounds storage even against a sorter making indefinitely many redundant queries; by then the desired comparison threshold has already been exceeded.

Algorithm 1 ANSWER(x, y)

```
if P already implies the comparison then
    return its implied answer.
end if
Set v to the root.
while x, y are both fixed at the same position c in v do
    Set v to the child at c.
end while
Test safety of x < y and y < x at v.
if one answer is unsafe then
    Choose the safe answer retaining every old assignment.
else
    Estimate P(X_x ≤ X_y) and P(X_y ≤ X_x) locally.
    Choose the answer with larger estimate.
end if
Add the chosen strict answer to P and its weak constraint at v.
Run width, degree, and path repairs at v.
return the chosen answer.
```

3 Legality and recursive independence

The auxiliary inequalities are weak, but the public answers are strict. Nonempty auxiliary counts alone would not establish legality: a strict directed cycle can still admit a constant weak assignment. We instead prove that the public order remains acyclic and that the recursive restrictions can always be realized together.

Lemma 3.1 (Effective support)**.** *The interval system is feasible exactly when $\ell_x \le r_x$ for every item. In a feasible system every integer in $[\ell_x, r_x]$ is attained by X_x. Along an active edge $x \to y$, $\ell_x \le \ell_y$ and $r_x \le r_y$.*

Proof. Every assignment respects propagated bounds. Conversely the lower-endpoint vector $(\ell_x)_x$ respects every interval and inequality if $\ell_x \le r_x$. To obtain $X_x = t$, add the bound $[t, t]$ at x and propagate again. No predecessor lower bound exceeds t, and no successor upper bound is below t, so the result is feasible. The final monotonicity statements follow from the definitions of the propagated endpoints. □

Lemma 3.2 (Interior fixing and endpoint insertion)**.** *Fixing $X_x = c$ with $\ell_x < c < r_x$ leaves at least two feasible positions for every other free item. The item x is publicly incomparable with all old items fixed at c. If a selected position is an endpoint, the insertion of Section 2.2 makes its new copy interior and retains at least as many assignments as the original endpoint fiber.*

Proof. Only predecessors of x can acquire a smaller upper bound after interior fixing; their lower bounds are at most $\ell_x < c$. Only successors can acquire a larger lower bound, and their upper bounds are at least $r_x > c$. Thus no other free interval becomes a singleton. An old fixed item p at c with $p <_P x$ would imply $X_x \ge c$ in every assignment, contradicting attainment of $\ell_x < c$. The case $x <_P p$ is symmetric.

For a lower endpoint c, take an old assignment with $X_x = c$. Raise x and exactly its successors currently equal to c to the new position c^+. No such successor is fixed, since a fixed successor at c would give $r_x \le c$. All inequalities survive, and every changed artificial interval admits the new position. Collapsing c^+ back to c recovers the original assignment,

proving injectivity. Assignments attaining the old lower and upper endpoints still exist before fixing; hence c^+ is interior. Reflect this argument for an upper endpoint.

It is essential that comparison-induced bounds are recomputed rather than enlarged as artificial bounds. A comparison $x < p$ with p fixed at the old identity c still implies $X_x \leq c$, not $X_x \leq c^+$. □

Lemma 3.3 (Independent admissions)**.** *The jth admission to a child multiplies its count by at least j. It changes neither the effective intervals nor the probability distribution of its old free items.*

Proof. Lemma 3.2 makes the incoming item publicly incomparable with every old item in that child. It therefore introduces no inequality there. Every old assignment has exactly as many extensions as there are positions in the incoming item's full interval, at least j. Appending a position does not enlarge any old interval, so this extension count is constant over old assignments. The assertion remains true if the child has answered comparisons between successive admissions. □

Lemma 3.4 (Safe routing)**.** *For a publicly incomparable pair a, b not both fixed at the same position, either both answers are safe, or a safe answer retains every old assignment. Delegation through a common fixed position changes only the count at the eventual stopping node and introduces no extra comparison into another child. Every resulting recursive state admits a strict total order extending P.*

Proof. Suppose adding $a < b$ creates a new public relation $i < j$ between two old items fixed at position c. There were old public paths $i \leq_P a$ and $b \leq_P j$. Consequently every old assignment satisfies

$$X_b \leq c \leq X_a. \tag{4}$$

The reverse weak answer retains everything. If both directions have such an obstruction, the two inequalities force $X_a = X_b = c$ throughout the old family. By Lemma 3.1 and the two-position invariant, both items were already fixed at that position: the delegation case.

Suppose instead that $a < b$ newly confines a free item w to c. Both bounds of w cannot have changed, since that would require an old path $b \leq_P w \leq_P a$, contrary to incomparability. If the lower bound rose, $b \leq_P w$, the old upper bound of w was c, and the old lower bound of a was c. This again gives (4). There was an old assignment with $X_w < c$, so there was a strict witness $X_b < X_a$. That witness excludes an inner-fixed-item obstruction to the reverse answer, since such an obstruction would force $X_a \leq X_b$ in every old assignment. The reverse cannot create a singleton either, because it retains the whole old family. A falling upper bound is symmetric. If a weak answer is infeasible, its reverse is strict in every old assignment, giving the same conclusion.

If both items are fixed at c, either weak answer retains every parent assignment. A newly acquired relation between items in this child is precisely a consequence of its requested comparison: its old path pieces are already in the child's public restriction. If another child at d acquired a relation, those pieces would imply $c \leq d \leq c$, so $d = c$. Thus delegation affects no other child.

Finally choose an admissible assignment at each node. Order different positions increasingly. Within a tied position, extend the restriction of P, respecting the order recursively chosen for its fixed child items. Any path through free items imposes only a relation already in that public restriction. A total order extending a restriction of a finite partial order can be extended to the larger set, so these choices can be combined. Publicly incomparable queries never create a directed cycle; the safety tests ensure consistency with the existing

child restrictions. Recursion therefore yields a strict total order on the original distinct items. □

The public order is the sorter's only information. Private interval cuts and fixings do not become extra answers. Consequently a correct sorter can stop only when P is total, even if it knows the adversary's program: it must also be correct on every other input giving the same transcript.

4 Why the numerical operations are efficient

Weakly order-preserving maps are classical counting objects [7]. Here the intervals and recursive admissions let us use them without counting the full public partial order. We need comparison probabilities, cumulative coordinate probabilities, and a reusable interval of positions with adequate point probabilities.

Lemma 4.1 (Central mass and anti-concentration)**.** *Suppose x has effective interval $[\ell, r]$, width W, and at most c other active predecessors and successors in total. Let $t_0 = \lfloor (W-1)/4 \rfloor$ and $I_x = [\ell + t_0, r - t_0]$. Then $|I_x| \geq W/2$ and*

$$\mathbb{P}(X_x = t) \geq \frac{1}{4^c W} \qquad (t \in I_x), \tag{5}$$

$$\mathbb{P}(X_x \in I) \leq \frac{2 \cdot 4^c |I|}{W} \qquad \text{for every integer interval } I. \tag{6}$$

For a different coordinate Y, even a dependent one, or a fixed real threshold Y,

$$\mathbb{P}(|X_x - Y| \leq z) \leq \frac{2 \cdot 4^c (2z+1)}{W} \qquad (z \geq 0). \tag{7}$$

Lemma 4.2 (Local approximation)**.** *Fix degree and height bounds d, h, a component width ratio $R \geq 1$, and an error $0 < \varepsilon < 1/4$. Explicit computable constants W_0, ρ have the following property. If every nontrivial active component has minimum width at least W_0 and maximum-to-minimum width ratio at most R, restricting to radius ρ around one or two requested coordinates, with their original propagated intervals, changes each requested weak comparison or cumulative probability by at most ε. Isolated coordinates can be treated exactly at any width.*

The proof in Appendix A couples two conditional-refresh experiments, one on the full graph and one on the truncated graph. These experiments are used only in the proof. The adversary's computation is deterministic and uses the following exact finite counter.

4.1 The subset counter

Let the restricted graph have vertex set V, $b = |V|$, and positions $1, \ldots, K$. A set is downward closed if it contains every predecessor of each of its members. In an admissible assignment, $S_t = \{x : X_x \leq t\}$ is downward closed, and $S_{t-1} \subseteq S_t$.

Initialize $C_0(\varnothing) = 1$ and the other entries to zero. For each $t = 1, \ldots, K$, and downward-closed $S \subseteq U \subseteq V$, perform

$$C_t(U) \mathrel{+}= C_{t-1}(S) \tag{8}$$

if the interval of every $x \in U \setminus S$ contains t. This assigns exactly those new items position t. No internal ordering is needed, since equal positions are permitted. The chain of sets S_t recovers the assignment uniquely; thus $C_K(V)$ is exactly the count.

There are at most 3^b pairs $S \subseteq U$: each item is outside U, in $U \setminus S$, or in S. Precomputing closure and transition conditions takes a polynomial factor in b. The counter uses at most $K3^b$ transitions on integers of $O(b \log K)$ bits. A comparison numerator adds the requested weak edge; a cumulative numerator adds an upper bound. Even if the added edge creates a weak directed cycle, downward closure forces all its vertices to enter together, correctly counting equality.

Use a single neighborhood for all cumulative thresholds. The resulting estimates then form the cumulative distribution of an actual local law and are automatically monotone. For a fixed endpoint c, the two answer probabilities are $F_x(c)$ and $1 - F_x(c-1)$. Since the two true weak probabilities sum to at least one, choosing the larger estimate retains at least $1/2 - 2\eta$. We use this conservative bound throughout.

Lemma 4.3 (An interval from approximate quartiles)**.** *Let $\widehat{F}$ be a local coordinate cumulative distribution with pointwise error at most $1/16$. Let a, b be its first crossings of $1/4, 3/4$. Then $[a, b]$ lies in the true effective support and*

$$\mathbb{P}(X_x = t) \geq \frac{1}{16(b - a + 1)} \qquad (a \leq t \leq b).$$

Appendix A.6 proves this using a counting injection establishing log-concavity of coordinate probabilities. A likely interval is needed here, rather than just a likely point: representative reuse is what controls the total cost of moves.

4.2 The width repairs meet the oracle hypotheses

Lemma 4.4 (Strict cuts and separated scales)**.** *Rule W2 retains at least $1 - \eta$ of the old count, preserves two-position supports, and implies the retired comparison after all future insertions. In W3 the selected items stay free with unchanged effective intervals and an incident edge until moved. Their representative choices retain a fraction at least*

$$\frac{1}{CVb(2 + \log_2(K/b))} \tag{9}$$

when chosen block size is b, with $V = 2 \cdot 4^B$ and an absolute C independent of J. At most two selected items enter any child.

Proof. If $W_u > LW_v$, endpoint monotonicity gives $r_u \leq r_v$. The part of u's interval rejected by $X_u \leq \ell_v - 1$ contains at most W_v positions. By (6) its probability is at most $C_* W_v / W_u < \eta$. The retained interval has length at least $W_u - W_v \geq 2$. Changed predecessor upper bounds remain strictly above their old lower bounds, since $\ell_v - 1 > \ell_u$. The artificial intervals on opposite sides of the cut are disjoint and ordered. Replacing a position by adjacent copies never destroys that strict separation. The other width direction is symmetric.

When W2 is unavailable, adjacent widths have ratio at most L. The item selected for scale i has width in $[A^i w, LA^i w]$. Consecutive selected widths have ratio at least $A/L > L^{2h}$. Even with shortcuts in the graph, they are therefore at distance greater than $2h$. Fixing an item affects only its active predecessors and successors, all within distance h, so it changes neither another selected item's interval nor its incident edges.

For each previous width at most w', a previously selected block has size at most w'. It forbids at most one block of the same kind in its batch. Thus previous choices exclude at most Jw' positions from the present central interval. The next width is greater than $32Jw'$, while its central interval contains at least half that width. After these exclusions and endpoint trimming a constant fraction remains. The largest-block estimate in Lemma 5.4 still applies with an absolute constant. The two kinds of representative permit at most two admissions at a position. □

Width repairs remove an edge or a free item, and therefore terminate. Afterwards every nontrivial component satisfies Lemma 4.2. Adding one new edge to a graph of height less than H joins at most two old directed paths; its height is less than $2H$. The degree rises by at most one. All ensuing repairs only remove vertices or edges. Thus the bounds d, h, B used above remain valid. All expensive numerical queries occur after width repairs.

5 Counting comparisons and paying for moves

We analyze a completed play with q comparisons. If $q > 2T$ the desired bound already holds, so assume $q \le 2T$. A cost is a decrease of $\log_2 Z$; an admission gain has negative cost. We keep these signs throughout. Charging a fixed positive amount for each move would lose the factorial cancellation that makes the argument work.

5.1 Population, incidence, and separation

For a node v, let s_v be the number of items ever entering it, and k_{vc} the number entering its child at c. Set

$$p_v = \sum_c k_{vc}, \qquad r_v = s_v - p_v, \qquad P_{\mathrm{mov}} = \sum_v p_v.$$

Here r_v counts the items that end free in v, and P_{mov} is the total number of moves.

Lemma 5.1 (Population and incidence)**.** *There are at most $2n-1$ nonempty nodes, and*

$$\sum_v r_v = n, \qquad \sum_v s_v = n + P_{\mathrm{mov}}, \qquad P_{\mathrm{mov}} \le 2q. \tag{10}$$

At most $K_v = 2s_v + 2$ positions are ever created in node v, and hence

$$K_{\mathrm{sum}} := \sum_v K_v \le 6n + 2P_{\mathrm{mov}}. \tag{11}$$

At most q active edges are retired.

Proof. Each item ends free at exactly one node and contributes one new node occurrence per move. This proves the first two identities. Every moved item has an incident comparison edge, either at movement or when a path containing it was reserved. Charge that move to the corresponding edge endpoint. An item leaves a node only once, so an edge receives at most two charges. Each edge came from an actual comparison at that node. This gives $P_{\mathrm{mov}} \le 2q$, bounds edge retirements, and also rules out an infinite repair sequence during a finite transcript.

The two endpoints of any charged edge cannot both enter the same child: the second would be publicly comparable with an old child item. A node with only one child therefore retains an item. So does a leaf. Those retained items are distinct across nodes. A finite rooted tree has fewer branching nodes than leaves, giving at most $2n-1$ nodes. In particular each child is a proper subset of its parent.

Initial positions and top-position additions create at most $s_v + 2$ positions, and at most one endpoint insertion occurs per moved item. Thus $K_v = 2s_v + 2$ is an upper bound. Summing proves (11). □

Define the separation budget

$$E_{\mathrm{sep}} = \sum_{v,c} k_{vc} \log_2 \frac{s_v}{k_{vc}}. \tag{12}$$

This quantity is used only in the analysis.

Lemma 5.2 (One separation budget)**.**

$$E_{\text{sep}} = T - \sum_v r_v \log_2 s_v \le T.$$

Proof. The contribution of v is $s_v \log_2 s_v - \sum_c k_{vc} \log_2 k_{vc} - r_v \log_2 s_v$. Every nonroot s_v equals the multiplicity on its incoming edge. The first two terms telescope, leaving only the root term T. □

Lemma 5.3 (Expensive moves are rare)**.** *Let $N_{\text{ord}}, N_{\text{sc}}$ count ordinary and scale moves. Then*

$$N_{\text{ord}} \le \frac{2q}{D-1} + \frac{T}{\log_2 H}, \qquad N_{\text{sc}} \le \frac{T}{\log_2(J/2)}. \tag{13}$$

Proof. A degree move has at least $D-1$ incident edges when selected. Charging all those endpoints and using the same incidence argument bounds their number by $2q/(D-1)$.

For destination counts k_c totaling p, partition the admissions into groups. In a group of size g, let t_c go to destination c. Concavity of entropy gives

$$\sum_{\text{groups}} \sum_c t_c \log_2 \frac{g}{t_c} \le \sum_c k_c \log_2 \frac{p}{k_c}. \tag{14}$$

This follows by applying concavity of $-x \log_2 x$ to the average of the groups' destination-frequency vectors, weighted by g/p.

All H items of a reserved path eventually move, even if some do so during width repair. They are publicly comparable and therefore enter distinct children. Each such group contributes $H \log_2 H$ to the left side of (14). The groups are disjoint at a node. The right side is bounded by its contribution to (12), since $p_v \le s_v$. Lemma 5.2 bounds the number of reserved path members by $T/\log_2 H$, and hence also bounds their ordinary moves.

Apply the same argument separately to scale groups. Each has J items and destination multiplicities at most two, so contributes at least $J \log_2(J/2)$. This proves the other bound. □

5.2 Representative reuse

Lemma 5.4 (A lifetime dictionary of representatives)**.** *Suppose at most K positions are ever born in a node. From an interval of m positions each having probability at least $1/(Vm)$, the representative rule choosing block size b retains at least the fraction* (9)*, with an absolute constant C. Over the node's lifetime, the number of different destinations selected with size b is at most*

$$\frac{K}{b}(2 + \log_2(K/b)). \tag{15}$$

The interval-size bound may be taken as $m \le 8b(2 + \log_2(K/b))$, including the restricted scale-group choices.

Proof. A consecutive subsequence of a batch has at most two maximal aligned blocks of any one size: each such block has a parent crossing one of the two endpoints. Thus blocks of size at most b cover fewer than $4b$ members of any batch. Batches smaller than b have fewer than b members in total, and there are at most $1 + \log_2(K/b)$ other current batches. Endpoint trimming retains a constant fraction of m, giving the stated bound. Intervals of size less than eight are covered by $b = 1$. The exclusions in a scale group remove less than a quarter of the central interval by the width ratios in Lemma 4.4; the same constant eight suffices.

At a fixed batch size B', at most K/B' batches ever occur, since their birth sets are disjoint. Each has B'/b aligned blocks of size b. Summing over $B' = b, 2b, \ldots$ gives (15). One additional K term at size one covers exceptional endpoint choices. □

5.3 Cheap moves and factorial credits

Credits can be separated into cheap, ordinary, and scale admissions. At the jth admission of one category, the child's total admission number is at least j. Thus k admissions of that category supply a factor at least $k!$, by Lemma 3.3.

For k cheap moves to one child, the net cost is at most

$$k \log_2 G - \log_2(k!) = \log_2 \frac{G^k}{k!} \le G \log_2 e,$$

because $G^k/k!$ is a term of $\exp(G)$. There are at most $2n-2$ children, and $2\log_2 e < 3$. Therefore

$$\text{net cheap cost} \le 3nG. \tag{16}$$

Cheap moves need not be rare.

5.4 The signed cost of the other moves

Put $X = N_{\text{ord}} + N_{\text{sc}}$ and $\sigma = 1 + 2B$, so $2 \cdot 4^B = 2^\sigma$. Set aside a surcharge σ for every scale move. Lemma 5.4 then bounds the remaining price of either kind of move by $\log_2[128b(2 + \log_2(K_v/b))]$. An ordinary move uses $V = 16$; the scale surcharge covers its larger V.

Lemma 5.5 (Total net move cost)**.** *All fixing costs minus all admission gains total at most*

$$3nG + \sigma N_{\text{sc}} + 40X \log_2 \left(2 + \frac{n + T + P_{\text{mov}}}{X}\right). \tag{17}$$

The last term is defined as zero if $X = 0$. The constant 40 is independent of every stage parameter.

Proof. Fix a node and one of the two expensive categories. For each destination c, let b_c be its largest selected block size in this category, and $j_c = \log_2(K_v/b_c)$. By (15), the positive weights

$$Q_c = \frac{b_c}{K_v(2 + j_c)^3} \quad \text{satisfy} \quad \sum_c Q_c \le 1.$$

Indeed, destinations with largest block size b contribute at most $(2 + \log_2(K_v/b))^{-2}$, and these arguments form a unit-spaced grid starting at a nonnegative number. Their sum is less than $\sum_{r \ge 0}((r+1)(r+2))^{-1} = 1$.

Let k_c be this category's multiplicity at c. The price increases with block size. Using $k! \ge (k/e)^k$ and $e < 4$, its net cost after the factorial credit is at most

$$k_c \log_2 \frac{512 b_c (2 + j_c)}{k_c}. \tag{18}$$

Discard negative terms and let I index the remaining node–destination–category triples, with $Y = \sum_I k_c \le X$. If $Y = 0$ there is nothing to bound. Positivity of (18) gives

$$k_c \le 512 K_v 2^{-j_c}(2 + j_c), \qquad j_c \le 2\log_2(K_v/k_c) + 22,$$

where the second estimate uses $\log_2(2 + j) \le j/2 + 2$.

For positive a_i, k_i, concavity of the logarithm gives

$$\sum_i k_i \log_2(a_i/k_i) \le Y \log_2 \left(\frac{\sum_i a_i}{Y}\right), \qquad Y = \sum_i k_i. \tag{19}$$

Since $b_c(2+j_c) = K_v Q_c (2+j_c)^4$ and there are two categories, summing (18) yields

$$\text{remaining cost} \le Y \log_2 \frac{1024 K_{\text{sum}}}{Y} + 4 \sum_I k_c \log_2(2+j_c). \tag{20}$$

Write k^*_{vc} for total admissions to that child, including all categories. Set $U = n+T+P_{\text{mov}}$. Since $K_v \le 4 s_v$,

$$\begin{aligned}\sum_I k_c \log_2(K_v/k_c) &= \sum_I k_c \log_2(K_v/s_v) + \sum_I k_c \log_2(s_v/k^*_{vc}) + \sum_I k_c \log_2(k^*_{vc}/k_c) \\ &\le 2Y + T + 2P_{\text{mov}}.\end{aligned}$$

The middle sum is bounded by Lemma 5.2, since the selected category multiplicities sum to at most k^*_{vc}. For the last sum use $x \log_2(1/x) \le 1$ on each of the two categories. By (19) and the bound on j_c,

$$\begin{aligned}\sum_I k_c \log_2(2+j_c) &\le Y \log_2 \left(24 + \frac{2}{Y} \sum_I k_c \log_2(K_v/k_c) \right) \\ &\le Y \log_2(28 + 4U/Y) \\ &\le 5Y + Y \log_2(2 + U/Y).\end{aligned}$$

Also $K_{\text{sum}} \le 6U$. Substitute these estimates in (20) to obtain

$$\text{remaining cost} \le 33Y + 5Y \log_2(2+U/Y) \le 40 Y \log_2(2+U/Y).$$

The function $y \log_2(2+U/y)$ increases for $y > 0$ and tends to zero as $y \downarrow 0$, so replace Y by X. Adding (16) and σN_{sc} proves the claim. □

The weights Q_c, maximum block sizes, and final multiplicities are proof devices. The online algorithm uses only the currently available blocks.

5.5 The terminal count and the comparison inequality

Lemma 5.6 (Terminal count)**.** *If the sorter has finished and $n \ge 2$, $q \le 2T$, then*

$$\log_2 Z_{\text{final}} \le n \log_2(21 + 24 \log_2 n).$$

Proof. The public order is total. The r_v free items at v must therefore occupy a nondecreasing sequence of positions in their known order. Ignoring additional bounds gives at most $\binom{K_v + r_v - 1}{r_v}$ assignments. Concatenate the node position lists in a fixed order. The tuple of node sequences injects into a nondecreasing sequence of $n = \sum_v r_v$ positions from the concatenated list: its pieces are recoverable from the known r_v. Consequently

$$Z_{\text{final}} \le \binom{K_{\text{sum}} + n - 1}{n} \le \left(\frac{e(K_{\text{sum}} + n)}{n} \right)^n.$$

Use $K_{\text{sum}} \le 6n + 2P_{\text{mov}}$, $P_{\text{mov}} \le 2q \le 4T$, and $e < 3$. □

Theorem 5.7 (Fixed-stage comparison bound)**.** *Every completed play with $n \ge 2$, $q \le 2T$ satisfies*

$$\begin{aligned}T \le (1+15\eta)q + n \log_2(21 + 24 \log_2 n) + 3nG + \sigma N_{\text{sc}} \\ + 40X \log_2 \left(2 + \frac{n + T + P_{\text{mov}}}{X} \right),\end{aligned} \tag{21}$$

where $P_{\text{mov}} \le 2q$ and (13) bounds $X = N_{\text{ord}} + N_{\text{sc}}$.

Proof. Each actual comparison retains either all assignments or at least $1/2 - 2\eta$ at one node. There are at most q strict edge cuts, each retaining at least $1 - \eta$. For $\eta \le 1/16$, their combined cost is at most $(1 + 15\eta)q$, using $-\ln(1 - z) \le 2z$ for $z \le 1/2$. Initially $\log_2 Z = T$. Endpoint insertions preserve old assignments, and Lemma 5.5 includes the signed admission credits. Telescope all changes and apply Lemma 5.6. □

6 One adversary with a fixed polynomial time bound

A family of fixed-stage algorithms attaining coefficients approaching one would not by itself prove Theorem 1.1. We must choose a stage effectively from n, with one polynomial exponent for all input sizes.

6.1 An explicit sequence of stages

For integers $j \ge 4$, choose

$$D = j + 4, \qquad H = 2^j, \qquad \eta = \frac{1}{100j}.$$

Compute d, h, B from (2), then set

$$\sigma = 1 + 2B, \qquad J = 2^{j\sigma+1}, \qquad L = \max\{4, \lceil C_*/\eta \rceil\}.$$

Compute A, R from (3), compute W_0, ρ using Appendix A, and put $G = 4^B W_0$. Each stage is a finite, explicitly computable list of integers and rationals.

For $q \le 2T$, Lemma 5.3 gives

$$X \le \frac{6T}{j}, \qquad \sigma N_{\mathrm{sc}} \le \frac{T}{j}, \qquad n + T + P_{\mathrm{mov}} \le 6T. \tag{22}$$

Monotonicity of the last term of (21) therefore gives

$$40X \log_2\left(2 + \frac{n + T + P_{\mathrm{mov}}}{X}\right) \le \frac{240 \log_2(j+2)}{j}\, T. \tag{23}$$

6.2 Size caps and bounded setup

At initialization, spend at most n bit-computation steps tabulating consecutive stages. Never use a partially computed stage. Accept a completed stage only when

$$G^2 \le \lfloor \log_2(n+1) \rfloor, \qquad 2(d+1)^{\rho+1} \le \lfloor \log_2(n+1) \rfloor. \tag{24}$$

Use the largest accepted stage, or the fixed-order fallback if there is none. Logarithmic upper roundings in the stage formulas can be computed using integer bit lengths. The tabulation has a literal step budget: very large intermediate computations may be interrupted without using their partial results. Thus setup itself has a uniform polynomial bound.

Every fixed stage takes finitely many steps and has finite constants. For all sufficiently large n it is both completed and accepted. Consequently the selected stage $j(n)$ tends to infinity. The square on G is needed:

$$\frac{3nG}{T} \le \frac{3\sqrt{\log_2(n+1)}}{\log_2 n} \longrightarrow 0.$$

6.3 Bit complexity

The second cap bounds the union of two query neighborhoods by $b \le \log_2(n+1)$ vertices. A node has at most $2n+2$ positions. The counter (8) therefore has at most $K3^b \le (2n+2)(n+1)^{\log_2 3}$ transitions, with a polynomial factor in b for their tests. Counts have $O(b \log K) = O(\log^2(n+1))$ bits. Scanning all positions to locate the approximate quartiles is still polynomial, as are exact rational comparisons of the resulting probabilities.

Reachability, interval propagation, components, and batch manipulation use polynomial-size data. Longest paths in an acyclic directed graph are found by dynamic programming in a topological order; no enumeration of paths of length H is required. A repair removes an edge or a free item and does not initiate a child repair. Before the query-count cutoff of Section 2.6, Lemma 5.1 bounds the number of moves, nodes, and stored comparisons polynomially in n. Afterwards a fixed total order suffices. These bounds give a single polynomial number of bit operations per answer.

Proof of Theorem 1.1. Legality follows from Lemma 3.4, and the preceding argument proves the time bound. For the comparison guarantee assume $q \le 2T$, since the other case is immediate. Divide (21) by T. The terminal remainder is $O(\log\log n/\log n)$; the cheap remainder tends to zero by (24); and (22)–(23) make the other repair remainders $O(\log(j(n)+2)/j(n))$. Also $\eta \to 0$. Thus $T \le (1+o(1))q + o(T)$, proving $q \ge (1-o(1))n\log_2 n$. □

The argument also applies to each fixed sequence of random choices of an always-correct sorter. This is a statement about interaction with an adaptive adversary; it does not assert a single fixed hard input independent of all possible random choices.

A Proof of the local counting and fixing lemmas

Throughout this appendix the active graph is acyclic and fixed items have been absorbed into propagated intervals. All laws are uniform on the finite set of admissible weak assignments. The random variables used in the proof are not sampled by the adversary.

A.1 Compression and insertion

Proof of Lemma 4.1. Replace stored intervals by effective intervals, which leaves the family unchanged. Delete x, keeping those intervals and all implied inequalities between remaining items. Let N^- count the resulting assignments and N the original ones.

Every deleted assignment admits an insertion of x: its predecessors lie before its successors, predecessor values are at most r, and successor values are at least ℓ. The allowed positions are a nonempty interval. There are at most W of them, so

$$N \le WN^-. \tag{25}$$

Transform a deleted assignment as follows. For each proper predecessor of x, leave values below ℓ unchanged and replace $\ell + i$ by $\ell + \lfloor i/4 \rfloor$ for $i \ge 0$. For each proper successor, leave values above r unchanged and replace $r - i$ by $r - \lfloor i/4 \rfloor$. Other coordinates are unchanged.

These transformations preserve all intervals: predecessor lower bounds are at most ℓ, and successor upper bounds at least r. They are nondecreasing within each class. No inequality enters the predecessor set from outside it, while lowering a predecessor helps an inequality leaving that set. The symmetric statement holds for successors. Thus every transformed assignment is admissible in the deleted system.

There are at most 4^c preimages of any transformed assignment. All transformed predecessors lie at or below $\ell + t_0$, and successors at or above $r - t_0$. Each position in I_x can therefore receive x. It follows that

$$N \geq 4^{-c}|I_x|N^- \geq \frac{WN^-}{2 \cdot 4^c}. \tag{26}$$

For a fixed $t \in I_x$, at least $4^{-c}N^-$ assignments have $X_x = t$. Divide by (25) to prove (5). An interval of m positions allows at most m insertions for each deleted assignment; divide mN^- by (26) to prove (6). With a different coordinate Y fixed in the deleted assignment, at most $2z + 1$ integer positions satisfy $|X_x - Y| \leq z$. This proves (7), including dependent coordinates and fixed real thresholds. □

A.2 Conditional refresh and a one-sweep bound

Take a topological order of the active graph. One sweep visits coordinates in that order and replaces each by a uniform allowed integer conditional on all other coordinates. Its allowed set is an interval $[a, b]$, and the replacement can be written, for a fresh uniform $U \in [0, 1)$,

$$\lfloor a + U(b - a + 1) \rfloor = \lfloor (1 - U)a + Ub + U \rfloor. \tag{27}$$

Each conditional update preserves the uniform assignment law. Thus starting in that law and performing any number of sweeps leaves it unchanged.

Compare full and truncated systems using the same fresh U at every common coordinate, and restrict the full topological order for the local sweep. Let Δ_x be the absolute discrepancy of their coordinate values. At a coordinate whose incident edges are present in both systems,

$$\Delta_x^{\text{new}} \leq 1 + (1 - U) \max_{y \to x} \Delta_y^{\text{new}} + U \max_{x \to z} \Delta_z^{\text{old}}. \tag{28}$$

Empty maxima are zero. The lower conditional endpoint is the maximum of its fixed interval bound and predecessor values; the upper endpoint is the minimum of its fixed interval bound and successor values. Equation (27) gives (28), with one unit for integer rounding.

Assume height at most h. Expand the predecessor terms backward through the sweep. Within at most h updates each predecessor-only path reaches a common fixed lower bound. Let A_x be the minimum of the fresh $1 - U$ values at x and all its active predecessors. The coefficient reaching such a common bound is at least A_x^h. At most h rounding errors accumulate, and any old discrepancy left in the expansion lies within undirected graph distance h of x. Consequently

$$\Delta_x^{\text{new}} \leq h + (1 - A_x^h) \max_{y \in B_h(x)} \Delta_y^{\text{old}}. \tag{29}$$

The minimum defining A_x covers every possible maximizing predecessor, so this inequality does not select a preferred path in advance.

A.3 Moment contraction in a shrinking interior

For maximum degree d, put

$$M = (d + 1)^{h+1}, \qquad a_0 = \frac{1}{8M^2}, \qquad s = \left\lceil \frac{8M}{a_0^h} \right\rceil. \tag{30}$$

Both $B_h(x)$ and the set of fresh numbers defining A_x have at most M members. A union bound gives $\mathbb{P}(A_x < a_0) \leq Ma_0$. On the complementary event $(1 - A_x^h)^s \leq e^{-sa_0^h}$, and hence

$$M\,\mathbb{E}(1 - A_x^h)^s \leq M(Ma_0 + e^{-sa_0^h}) \leq \frac{1}{2}. \tag{31}$$

Retain the radius-ht neighborhoods of the requested coordinates. Start each system in its own uniform law, with any initial coupling, and run t coupled sweeps. At sweep j restrict attention to S_j, the union of the radius-$h(t-j)$ neighborhoods of the requested coordinates. For $x \in S_j$, $B_h(x) \subseteq S_{j-1}$, so all edges needed for (29) are present.

Writing $z_+ = \max\{z, 0\}$, (29) implies

$$(\Delta_x^{\text{new}} - hj)_+ \le (1 - A_x^h) \max_{y \in B_h(x)} (\Delta_y^{\text{old}} - h(j-1))_+.$$

The fresh numbers are independent of the old discrepancies. Raise to the power s, bound a maximum by the sum of the sth powers, and apply (31). The largest moment in the shrinking interior falls by a factor at least two per sweep. Initially, both values at a common coordinate lie in its same globally propagated interval. If $W_{\max,x}$ is the largest width in x's full component, induction gives at a requested coordinate

$$\mathbb{E}(\Delta_x - ht)_+^s \le W_{\max,x}^s 2^{-t}. \tag{32}$$

No assumption about the truncated boundary law has been made.

A.4 Explicit radius and minimum width

There are at most $2M$ other coordinates comparable with one coordinate. For a desired error ε, set

$$C_{\text{ins}} = 2 \cdot 4^{2M}, \qquad u = \frac{\varepsilon}{16 C_{\text{ins}}}. \tag{33}$$

Choose an integer t and integer W_0 satisfying

$$t \ge s \log_2(2R/u) + \log_2(8/\varepsilon), \tag{34}$$

$$W_0 \ge \max\{32,\ 2ht/u,\ 4C_{\text{ins}}/\varepsilon\}. \tag{35}$$

Take $\rho = ht$. These choices are computable with integer upper roundings.

For a nonisolated requested coordinate, $W_{\max,x} \le RW_x$ and $ht \le uW_x/2$. Markov's inequality and (32) give

$$\mathbb{P}(\Delta_x > uW_x) \le (2R/u)^s 2^{-t} \le \varepsilon/8. \tag{36}$$

Outside these exceptional events, disagreement between the two weak comparison indicators requires $|X_x - X_y| \le u(W_x + W_y)$. Applying (7) at the larger-width endpoint, of width W, bounds this probability by

$$4C_{\text{ins}}u + C_{\text{ins}}/W \le \varepsilon/2.$$

The two exceptional probabilities leave a total less than ε. A fixed threshold uses the same argument with one requested coordinate. An isolated coordinate is exactly uniform and independent of all others: couple its values identically. If both coordinates are isolated the local answer is exact; if one is nonisolated the larger-width endpoint in (7) still has width at least W_0. Both experiments preserve their uniform laws, so these disagreement bounds prove Lemma 4.2.

A.5 Log-concavity by an injection

Let $p_i = \mathbb{P}(X_x = i)$. We prove

$$p_{i-1} p_{i+1} \le p_i^2. \tag{37}$$

Take admissible assignments f, g with $f(x) = i - 1$, $g(x) = i + 1$, and write $h = g - 1$ coordinatewise. Both f and h respect all weak edges, although h may violate interval

bounds. On the items form an undirected graph containing the edge of an inequality $y \to z$ exactly when

$$\max\{f(y), h(y)\} > \min\{f(z), h(z)\}. \tag{38}$$

Across such an edge $h - f$ is nonzero and has the same sign at both ends. For example, if $h(y) > f(y)$ and $h(z) \le f(z)$, then $h(y) \le h(z)$ contradicts (38). The other sign change is symmetric.

On the component containing x we therefore have $h > f$. Swap f, h on that component, then add one to the second assignment. Both output assignments take value i at x. For an item y in the component, $\ell_y \le f(y) < h(y) \le r_y - 1$, so the swapped values, with the indicated addition, obey its original interval. An edge crossing the component boundary fails (38), so either choice of its two values preserves the inequality. Other edges are unchanged or swapped together. The outputs are therefore admissible.

The map is injective. Subtract one from the second output: the unordered coordinate pairs are precisely the original $\{f(y), h(y)\}$. They determine (38) and its component at x. Swap back and add one to recover g. Counting the input and output pairs proves (37).

By Lemma 3.1, positive probabilities occupy an interval. The ratios p_{i+1}/p_i decrease there. The cumulative sums F_i are log-concave too: for $k \le i-1$, $p_k p_{i+1} \le p_{k+1} p_i$. Summing gives $F_{i-1} p_{i+1} \le F_i p_i$, equivalently $F_{i-1} F_{i+1} \le F_i^2$. Reflection gives the same property for inclusive tails. Zero terms at the endpoints cause no difficulty.

A.6 Approximate quartiles give a reliable interval

Proof of Lemma 4.3. Since $\|\widehat{F} - F\|_\infty \le 1/16$ and a, b are first crossings,

$$\begin{aligned} F(a) &\ge 3/16, \quad F(a-1) < 5/16, \\ F(b) &\ge 11/16, \quad F(b-1) < 13/16. \end{aligned}$$

Put $m = b - a + 1$. If $F(a-1) < 1/8$, then $p_a > 1/16$. Otherwise log-concavity of F implies

$$\frac{11}{5} \le \frac{F(b)}{F(a-1)} \le \left(\frac{F(a)}{F(a-1)}\right)^m,$$

and therefore

$$p_a \ge \frac{1}{8}\left((11/5)^{1/m} - 1\right) \ge \frac{\ln(11/5)}{8m} > \frac{1}{16m}.$$

The reflected argument using inclusive tails gives $p_b \ge 1/(16m)$. For $m > 1$, log-concavity of p gives, for $0 \le r < m$,

$$p_{a+r} \ge p_a^{1-r/(m-1)} p_b^{r/(m-1)} \ge \min\{p_a, p_b\}.$$

The case $m = 1$ already follows from the endpoint bound. The local coordinate law stays within the original effective interval, whose full support is attained globally, so its crossings lie there. □

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. An adversary-based lower bound for sorting. *Information Processing Letters*, 13(2):55–57, 1981. doi: 10.1016/0020-0190(81)90031-4.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C.-C. Yao. Efficient searching using partial ordering. *Information Processing Letters*, 12(2):71–75, 1981. doi: 10.1016/0020-0190(81)90005-3. URL `https://www.cs.toronto.edu/~bor/Papers/efficient-searching-using-partial-ordering.pdf`.

[3] Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8(3):225–242, 1991. doi: 10.1007/BF00383444.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. The randomized complexity of maintaining the minimum. *Nordic Journal of Computing*, 3(4):337–351, 1996. URL `https://www.brics.dk/RS/96/40/BRICS-RS-96-40.pdf`.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. A sort of an adversary. In *Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms*, pages 1291–1310. SIAM, 2019. doi: 10.1137/1.9781611975482.79. URL `https://doi.org/10.1137/1.9781611975482.79`.

[6] Dana Richards and Pravin Vaidya. On the distribution of comparisons in sorting algorithms. *BIT*, 28:764–774, 1988. doi: 10.1007/BF01954896. URL `https://lic225.github.io/vaidya/sorting_BIT88.pdf`.

[7] Richard P. Stanley. Two poset polytopes. *Discrete & Computational Geometry*, 1:9–23, 1986. doi: 10.1007/BF02187680. URL `https://math.mit.edu/~rstan/pubs/pubfiles/66.pdf`.